,

Problem A. Distances

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Three integers a, b, and c were chosen.

These three numbers are unknown to you, but instead the values $x = |a - b|$, $y = |b - c|$, and $z = |c - a|$ were computed. Recall that

$$|t| = \begin{cases} t & (t \geq 0) \\ -t & (t < 0) \end{cases}$$

Given x, y, and z, find any integers a, b, and c from which they could have been obtained.

It is guaranteed that the required a, b, and c definitely exist and lie in the interval from -10^8 to 10^8.

Input

The input consists of three lines, each containing one integer — x, y, and z, respectively ($0 \leq x, y, z \leq 10^8$).

Output

Print three integers a, b, and c for which the required equalities hold, each on its own line. The condition $-10^8 \leq a, b, c \leq 10^8$ must be satisfied.

Examples

standard input	standard output
1 2 3	3 2 0
0 0 0	5 5 5

Problem B. Minimize the Imbalance

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given an array a of n integers. Let the *imbalance* of this array be the maximum absolute difference between adjacent elements, that is,

$$D = \max_{i=1}^{n-1}(|a_i - a_{i+1}|).$$

You are allowed (but not required) to change **exactly one** a_i to another integer. The goal is to make the value of the imbalance D as small as possible.

Determine which a_i should be changed in order to achieve the minimum possible value of the imbalance.

Input

The first line of input contains a single integer n — the length of the array ($2 \le n \le 5 \cdot 10^5$).

The second line contains n integers a_i separated by spaces — the elements of the array ($1 \le a_i \le 10^9$).

Output

Print three integers separated by spaces: $D_{\min}$, i, and a_i^* — the minimum imbalance value that can be achieved, as well as the position of the modified element and its new value ($1 \le i \le n$; $1 \le a_i^* \le 10^9$).

If there are multiple possible answers, print any of them. In particular, if $D_{\min}$ is equal to the initial value D, you may print any i and $a_i^* = a_i$.

Examples

standard input	standard output
5 4 1 3 5 4	2 2 3
4 1 2 1 1	0 2 1

Problem C. Matrix Puzzle

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given a table with n rows and m columns and two allowed operations on it:

- shift the rows of the table down by one (the last row moves to the very top);
- shift the columns of the matrix right by one (the rightmost column moves to the leftmost position).

The *optimality* of a matrix is computed as follows: for each pair of adjacent cells, compute their product, and then sum all these products:

$$\mathtt{X} = \sum_{i=1}^{n} \sum_{j=1}^{m-1} a_{i,j} a_{i,j+1} + \sum_{i=1}^{n-1} \sum_{j=1}^{m} a_{i,j} a_{i+1,j}.$$

For example, consider the following matrix:

$$\begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{matrix}$$

Its optimality is $1 \cdot 2 + 2 \cdot 3 + 4 \cdot 5 + 5 \cdot 6 + 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 90$. If we apply the second operation, the matrix becomes $[[3, 1, 2], [6, 4, 5]]$, with optimality $3 \cdot 1 + 1 \cdot 2 + 6 \cdot 4 + 4 \cdot 5 + 3 \cdot 6 + 1 \cdot 4 + 2 \cdot 5 = 81$.

Determine the maximum possible value of the matrix optimality that can be obtained if you are allowed to perform any number of operations.

Input

The first input line contains two integers n and m separated by spaces — the dimensions of the matrix ($2 \le n, m \le 500$).

The next n lines contain m numbers each and describe the initial state of the matrix. All numbers in the matrix are between 0 and 1000, inclusive.

Output

Output a single number — the maximum possible value of the optimality.

Examples

standard input	standard output
2 3 1 2 3 4 5 6	95
3 3 9 1 1 1 6 1 1 1 3	58

Problem D. Step-by-Step Assembly

Input file:	standard input
Output file:	standard output
Time limit:	2 seconds
Memory limit:	256 megabytes

You want to assemble a certain mechanism. To do this, you have an unlimited amount of scrap metal and three conveyor belts. Moreover,

1. the first conveyor can produce one part of type I from scrap metal in t_1 time;
2. the second can produce one part of type II from one already finished part of type I in t_2 time;
3. the third can produce one part of type III from one part of type I **and** one part of type II in t_3 time.

The first conveyor is optimized quite well: it is guaranteed that $1 \leqslant t_1 \leqslant 2$.

The conveyors can work in parallel, but each individual conveyor can work on at most one part at a time (that is, no conveyor can start producing the next part before finishing the previous one).

It is known that the mechanism requires n parts of type III. What is the minimum possible time needed to produce these parts?

Input

The first line of input contains a single integer n — the number of required mechanisms ($1 \leq n \leq 1000$).

The second line contains three integers t_1, t_2, and t_3 separated by spaces — the time needed to produce parts of types I, II, and III, respectively ($1 \leq t_1 \leq 2$; $1 \leq t_2, t_3 \leq 10^5$).

Output

Print a single integer — the minimum time required to produce n mechanisms.

Examples

standard input	standard output
1 1 5 6	12
2 2 1 4	12
10 1 7 20	208

Note

In the first example:

1. The first part of type I is ready at time 1, after which the second conveyor immediately starts producing a part of type II.
2. By time 6, the first conveyor will produce 5 more parts of type I, and the second conveyor will finish its part.
3. One part of type I and one part of type II can immediately be sent to the third conveyor, and the mechanism will be ready at time 12.

Problem E1. Cutting the Board

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Two players play on an $n \times m$ grid board. At the beginning of the game, a token is glued at the intersection of the first (topmost) row and the first (leftmost) column.

The players take turns making moves. On each move, the current player chooses any horizontal or vertical strip between two rows or columns and makes a straight cut. The board splits into two non-empty rectangular parts, one of which contains the glued token and the other does not. Then the part without the token is removed from the game.

The part remaining in the game is never rotated or flipped. Rows and columns are numbered starting from one, from left to right and from top to bottom, beginning at the top-left corner.

A player who has no move loses, that is, the player whose turn comes when it is impossible to cut the board into two rectangular parts along the lines between cells. Your task is to determine which player has a winning strategy and play for that player.

Input

The only input line contains two integers n and m — the board dimensions ($1 \le i \le n \le 10^5$).

Interaction Protocol

This is an interactive problem.

As the first action in the interaction, your program must print on a separate line the number of the player you are going to play for: `1` or `2`.

Then the game begins: if you chose to play for the first player, your program must output the first move. Otherwise, the interactor makes the first move. A move consists of printing a line "`h` i" or "`v` j", denoting a horizontal cut between rows i and $i + 1$ or a vertical cut between columns j and $j + 1$, respectively.

The interactor may also print:

- `FAIL`, if your program makes an invalid move;
- `YOU WIN`, if your opponent has no move.

In both cases, your program must terminate immediately with exit code zero to avoid the Time Limit Exceeded and Idleness Limit Exceeded verdicts. If your program cannot make a valid move, print any invalid move, for example, "`0 0`".

After each printed line, you must flush the output buffer (`sys.stdout.flush()` in Python, `System.out.flush()` in Java, `cout.flush()` in C++).

,

Examples

standard input	standard output
3 3 h 1 YOU WIN	 2 v 1
4 5 v 1 YOU WIN	 1 h 2 h 1

Problem E2. Cutting the Board

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Two players play on an $n \times m$ grid board. At the beginning of the game, $1 \le i \le n$ and $1 \le j \le m$ are chosen, after which a token is glued at the intersection of the i-th row and the j-th column.

The players take turns making moves. On each move, the current player chooses any horizontal or vertical strip between two rows or columns and makes a straight cut. The board splits into two non-empty rectangular parts, one of which contains the glued token and the other does not. Then the part without the token is removed from the game.

The part remaining in the game is never rotated or flipped. Rows and columns are numbered starting from one, from left to right and from top to bottom, beginning at the top-left corner.

A player who has no move loses, that is, the player whose turn comes when it is impossible to cut the board into two rectangular parts along the lines between cells. Your task is to determine which player has a winning strategy and play for that player.

Input

The only input line contains four integers n, m, i, and j separated by spaces — the board dimensions and the position of the token chosen at the start of the game ($1 \le i \le n \le 10^5$; $1 \le j \le m \le 10^5$).

Interaction Protocol

This is an interactive problem.

As the first action in the interaction, your program must print on a separate line the number of the player you are going to play for: `1` or `2`.

Then the game begins: if you chose to play for the first player, your program must output the first move. Otherwise, the interactor makes the first move. A move consists of printing a line "`h` i" or "`v` j", denoting a horizontal cut between rows i and $i + 1$ or a vertical cut between columns j and $j + 1$, respectively.

The interactor may also print:

- `FAIL`, if your program makes an invalid move;
- `YOU WIN`, if your opponent has no move.

In both cases, your program must terminate immediately with exit code zero to avoid the Time Limit Exceeded and Idleness Limit Exceeded verdicts. If your program cannot make a valid move, print any invalid move, for example, "`0 0`".

After each printed line, you must flush the output buffer (`sys.stdout.flush()` in Python, `System.out.flush()` in Java, `cout.flush()` in C++).

,

Examples

standard input	standard output
3 3 2 2 h 1 h 1 YOU WIN	 2 v 1 v 1
4 5 3 2 v 4 v 2 v 1 YOU WIN	 1 h 3 v 1 h 1 h 1

Problem F. XOR Reduction

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	256 megabytes

You are given an array of integers a of length n.

In one operation, you may choose two elements a_i and a_j, remove them, and add a new element with value $a^*_{i,j} = a_i \oplus a_j$, where $\oplus$ denotes the bitwise exclusive OR operation (also written as `xor` or `^`).

There are only two restrictions on this operation:

1. the two original elements of the array are removed permanently and can no longer participate in any further operations;
2. any value obtained by such an operation, $a_{i,j}$, must remain in the array until the very end, that is, it also cannot participate in any further operations.

In other words, you may choose several disjoint pairs of elements, remove all of them, and replace them with the bitwise exclusive OR of the elements inside each pair. In the end, you obtain a new array consisting of all newly obtained numbers, as well as all original elements that did not participate in any operation.

Your goal is to minimize the maximum element of the final array by applying an arbitrary number of operations.

Input

Each test consists of several sets of input data. The first line of the input contains a single integer t — the number of sets of input data in the test ($1 \le t \le 50$). The sets of input data follow.

The first line of each set of input data contains an integer n — the initial length of the array ($1 \le n \le 50\,000$). It is guaranteed that the sum of n over all sets of input data does not exceed 10^5.

The second line contains n integers a_i — the elements of the array ($0 \le a_i \le 10^9$).

Output

For each set of input data, output a single integer on a separate line — the minimum possible value of the maximum among all elements of the final array.

Example

standard input	standard output
4 3 1 2 3 7 1 4 9 16 25 36 49 6 0 1 2 5 6 7 10 5 7 14 10 12 2 1 13 3 11	1 21 4 8

Note

In the first example, it is beneficial to replace 2 and 3 and obtain $a_{2,3} = 1$.

In the second example, 16 and 25 are replaced with $a_{4,5} = 9$, and 36 with 49 are replaced with $a_{6,7} = 21$.

In the third example, it is beneficial to choose the pairs $2, 6$ and $5, 7$.

Problem G. String Mutation

Input file:	`standard input`
Output file:	`standard output`
Time limit:	4 seconds
Memory limit:	1024 megabytes

You have string p of length n, each of its characters is from '`a`' to '`z`', in other words a lowercase Latin letter.

In one action you can:

- choose any number i from 1 to n and change the value of p_i to any other c from '`a`' to '`z`';
- choose two possible values c_1 and c_2 from '`a`' to '`z`' and replace the value of **all** characters equal to c_1 with c_2;
- check if strings formed by the characters of the string on positions from l_1 to r_1 and from l_2 to r_2 (where $l_1 \le r_1$ and $l_2 \le r_2$) are equal; i.e. that the following holds:

$$\begin{cases} r_1 - l_1 = r_2 - l_2 \\ p_{l_1+i} = p_{l_2+i} \qquad \text{for all } 0 \le i \le r_1 - l_1 \end{cases}.$$

Determine for each query of the third type whether the corresponding strings are equal or not.

Input

The first line of input contains a string p of length n, consisting of lowercase Latin letters — the initial characters ($1 \le n \le 2 \cdot 10^5$).

The next line contains a single integer q — the number of actions that will be performed ($1 \le q \le 2 \cdot 10^5$).

In the following q lines, the descriptions of the queries are listed in the form:

- "`1` i c" — assign the value c to the i-th character of the string ($1 \le i \le n$; '`a`' $\le c \le$ '`z`');
- "`2` c_1 c_2" — replace all values of all characters in the string equal to c_1 with c_2 ('`a`' $\le c_1, c_2 \le$ '`z`');
- "`3` l_1 r_1 l_2 r_2" — check that substrings formed by segments $[l_1, r_1]$ and $[l_2, r_2]$ are equal ($1 \le l_1, r_1, l_2, r_2 \le n$; $l_1 \le r_1$; $l_2 \le r_2$).

Output

For each query of the third type, output on a separate line a string "`YES`" (without quotes) if corresponding substrings are equal, and "`NO`" otherwise.

,

Examples

standard input	standard output
aaabc 6 3 1 2 2 3 1 4 c 2 c a 3 1 4 2 5 1 3 x 3 1 4 2 5	YES YES NO
abracadabra 7 3 1 4 8 11 1 7 r 2 a c 2 b c 3 1 4 8 11 3 1 4 5 8 3 2 4 6 8	YES YES YES YES

Problem H. Broken Clock

Input file:	standard input
Output file:	standard output
Time limit:	2 seconds
Memory limit:	256 megabytes

You received a broken clock as a gift. The clock consists of a dial with $60 \cdot 12 \cdot t$ marks, numbered clockwise from 0 to $720t - 1$, as well as a minute hand and an hour hand.

The hands move at the usual speed: the hour hand makes a full revolution of the dial in 12 hours, and the minute hand in 1 hour. The movement is discrete rather than continuous; in other words, once every fixed time interval, called a *tick*, each hand instantly jumps between marks on the dial. The parameter t determines the precision of the clock: t ticks per minute. In one such tick, the minute hand instantly moves forward by 12 marks, and the hour hand by one mark.

Only the minute hand is broken: when on the next tick it is supposed to catch up with or overtake the hour hand, it instead jumps back to the beginning, to mark number 0. More formally, if the minute hand points to mark m, and the hour hand to mark h, and the distance between them $(h - m) \bmod 720t$ is **strictly less** than 12, then on the next tick the minute hand will jump to mark 0.

You need to answer q queries. Each query asks you to determine how much real time it will take for the hands to move from state $s_{i,1}$ to state $s_{i,2}$.

Input

The first line of the input contains two integers t and q — the precision of the clock and the number of queries ($1 \le t \le 1500$; $1 \le q \le 5 \cdot 10^5$).

The i-th of the next q lines contains the description of the i-th query: four integers h_1, m_1, h_2, and m_2 — the numbers of the marks pointed to by the hour and minute hands in the initial and final states, respectively ($0 \le h_{1,2}, m_{1,2} < 720t$).

Output

For each of the q queries, output on a separate line the minimum number of ticks required for the clock to move from the first given state to the second, or `-1` if the second state is unreachable from the first.

Examples

standard input	standard output
1 4 0 0 1 12 0 0 60 0 11 0 12 0 12 0 13 0	1 60 1 -1
2 5 1201 1317 588 396 658 1196 102 84 442 8 1246 1200 79 0 739 0 355 286 98 72	827 -1 -1 660 5503